PROGRAMA EDUCATIVO EN:

Tecnologías de la Información en Desarrollo y Gestión de Software

DESARROLLO WEB PROFESIONAL

NOMBRE DEL ESTUDIANTE:

Héctor Ulises Cacho González

Dana Lizbeth Castañeda Sánchez

Oscar Yael Hernández Rodríguez

Concepción Guadalupe Paniagua González

Ciro Julián Cervantes Zamora

S - 5. DOM Y COMPONENTES DINÁMICOS ACCESIBLES

8VO CUATRIMESTRE “A”

Gestión del DOM y Estrategias de Optimización en React

DOM

El DOM representa la estructura de una página web como un árbol de nodos. Históricamente, la manipulación directa del DOM ha sido el principal factor de degradación de rendimiento debido a:

- Costo de Reflow: El cálculo de geometrías y posiciones del diseño.
- Costo de Repaint: La actualización visual de los elementos en pantalla.
- Inestabilidad: La manipulación imperativa suele derivar en estados inconsistentes de la UI.

El Virtual DOM (VDOM)

React implementa una abstracción ligera del DOM en memoria. El proceso se divide en tres fases críticas:

1. Renderizado: Generación de una nueva representación virtual tras un cambio de estado.
2. Reconciliación (Diffing): Algoritmo de comparación con complejidad $O(n)$ que identifica las diferencias mínimas.
3. Compromiso (Commit): Aplicación de cambios al DOM real de forma atómica y agrupada (*batching*).

Justificación de la Gestión Interna

La decisión de React de manejar el DOM internamente responde a la necesidad de Declaratividad. Al abstraer la manipulación de nodos, el desarrollador se enfoca en el "qué" (el estado de los datos) y no en el "cómo" (instrucciones manuales para el navegador).

Identificación de Errores Críticos

Ciclos de Re-renderizado Ineficientes

La creación constante de nuevas referencias (objetos o funciones) durante el renderizado provoca que componentes descendientes se actualicen innecesariamente.

- Solución: Implementación de useCallback para funciones y useMemo para valores calculados.

Degradación de la Experiencia de Usuario (UX)

La pérdida de foco en elementos de entrada es un error común que rompe la interactividad.

- Causa: Re-montaje del componente debido a una key inestable.

Accesibilidad Digital y el uso de ARIA

El uso de ARIA (Accessible Rich Internet Applications) es fundamental para que el Virtual DOM sea semánticamente útil para usuarios que utilizan lectores de pantalla o tecnologías asistivas.

Integración de ARIA en React

A diferencia de otros atributos que cambian a *camelCase* en React (como className o onClick), los atributos ARIA mantienen el formato de guiones idéntico al HTML estándar (ej. aria-label, aria-expanded).

Errores Comunes de Accesibilidad

1. Valores Estáticos vs Dinámicos: Olvidar vincular el estado de React con el atributo ARIA. Por ejemplo, un menú desplegable debe alternar aria-expanded="true" o "false" según su estado real.
2. Uso de Roles Redundantes: Aplicar role="button" a un elemento <button>. Esto es innecesario ya que el elemento nativo ya posee ese rol semántico.
3. Etiquetado Insuficiente: No proporcionar aria-label en botones que solo contienen iconos (como una "X" para cerrar).

Atributos Críticos para el Desarrollo en React

- aria-live: Permite que el lector de pantalla anuncie cambios dinámicos en el DOM sin que el usuario pierda el foco actual (ideal para notificaciones o carga de datos).
- aria-controls: Establece una relación lógica entre un elemento controlador (un botón) y el elemento que controla (una lista).

Referencias Bibliográficas

Abramov, D. (2019, 20 de mayo). Before you memo(). Overreacted. https://overreacted.io/before-you-memo/

Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Apps (2nd ed.). O'Reilly Media.

Meta Platforms, Inc. (s.f.). Accessibility (ARIA). React Documentation. Recuperado el 8 de febrero de 2026, de https://www.google.com/search?q=https://react.dev/reference/react-dom/components/common%23accessibility-attributes

Meta Platforms, Inc. (s.f.). Preserving and Resetting State. React Documentation. Recuperado el 8 de febrero de 2026, de https://react.dev/learn/preserving-and-resetting-state

W3C. (2023). WAI-ARIA Overview. Web Accessibility Initiative. https://www.w3.org/WAI/standards-guidelines/aria/

Zakas, N. C. (2016). Understanding ECMAScript 6: The Definitive Guide for JavaScript Developers. No Starch Press